

3강 파이썬 프로그래밍 2



컴퓨팅사고와 데이터 분석 기초

한영신

공지사항

저희 수업을 비롯해 모든 수업의 강의영상은 해당 영상을 만들어서 업로드한 분에게 저작권이 있습니다. 강의영상을 무단으로 링크를 배포하거나 수업 외의 사용을 금합니다.

본 사이트에서 수업자료로 이용되는 저작물은 저작권법 제 25조 수업목적저작물 이용 보상금제도에 의거, 한국복제전송저작권협회와 약정을 체결하고 적법하게 이용하고 있습니다. 약정범위를 초과하는 사용은 저작권법에 저촉될 수 있으므로 수업자료의 대중 공개 공유 및 수업 목적외의 사용을 금지합니다. 해당 규정을 어기는 학생은 법적으로 처벌의 대상이 될 수 있음을 유의해 주시기 바랍니다.

본 강의교안의 저작권은 한영신입니다.

이 자료는 강의 보조자료로 제공되는 것으로 무단으로 전제하거나 배포하는 것을 금합니다 .



CONTENTS



- 01 비교 & 불리언
- 02 조건문
- 03 반복문
- 04 반복문 심화

01

비교 & 불리언



비교 & 불리언

➤ 비교

프로그래밍에서의 비교는 두 대상을 비교하는 것을 말한다.

두 대상이 같은 값인지, 다른 값인지, 큰 값인지, 작은 값인지를 비교한다.

➤ 불리언(Boolean)

논리값인 참(True), 거짓(False) 중 한 가지를 가진다.

정수(int)나 문자(String)과 같은 하나의 데이터 타입이다.

비교 & 불리언 __ 01

비교 & 불리언

참 거짓

TRUE FALSE

Boolean

명제

실습

a=1

b=1

print(a==b)

print(1==2)

print(1>2)

print(1<2)

print(True)

print(False)

조지 불(영어: George

Boole IPA: [dʒɔ:(j)dʒ

bu:l], 1815년 11월

2일 ~ 1864년 12월

8일)은 영국의 수학자, 논리학자

이다. 논리대수인 불 대수를

창안해, 기호논리학 분야에 큰

업적을 남겼다.



클로드 엘우드 새넨(영어: Claude

Elwood Shannon, 1916년 4월

30일 ~ 2001년 2월 24일)은 미국의

수학자이자 전기공학자이다. 정보

이론의 아버지라고 불리며, 그가

작성한 《A Mathematical Theory of

Communication》 논문은 정보

이론의 시초가 되었다. 또한 불

논리를 전기회로로 구현할 수 있는

방법을 발명하여, 디지털 회로

이론을 창시하였다.



출처 : 위키피디아

비교 & 불리언 __ 01

02 조건문



조건문

조건문 __ 02

▶ 조건문

하나의 조건에 근거하여 다음에 실행할 처리를 결정 하는 수행문

내일 비가 오면
조건



축구경기는 취소된다
실행



➤ 관계 연산자

두 개의 피연산자를 비교 하는 연산자를 관계 연산자라고 한다.

관계 연산자	뜻
$x > y$	x가 y보다 큰가요?
$x < y$	y가 x보다 큰가요?
$x \geq y$	x가 y보다 크거나 같나요?
$x \leq y$	y가 x보다 크거나 같나요?
$x == y$	x와 y는 같나요?
$x \neq y$	x가 y다른가요?

➤ 관계 연산자 결과 값

관계 연산자를 사용한 수식은 **True(참)** 과 **False(거짓)** 의 두 가지 결과 값을 생성한다.

True(참)

1 < 10

9 <= 10

10 != 100

500 == 500

False(거짓)

1 > 10

9 >= 10

10 == 100

500 != 500

조건문

조건문 __ 02

➤ if-else

조건

만약 `if age >= 19 :`

콜론 (:) 의 의미 -
'문장의 끝이 아니다.'

`print("성인입니다 . ")` 조건이 `true( 참 )` 일 경우

그렇지
않으면 `else :`

`print("성인이 아닙니다.")` 조건이 `false( 거짓 )` 일 경우

만약 나이 (age) 가 19 세 이상이면 성인 , 그렇지 않으면 성인이 아니다 .

조건문

조건문 __ 02

➤ 관계 연산자 실습하기

나이가 19 세 이상이면 '성인입니다.' 19세 이상이 아니면 '성인이 아닙니다.'

```
age = int(input( 나이를 입력하세요 : "))
if age >= 19 :                                # 조건 ( age >= 19 )
    print("성인입니다 .")                    # 입력한 나이가 19 세 이상일 경우
else :
    print("성인이 아닙니다 .")              # 입력한 나이가 19 세 미만일 경우
```

```
Python 3.5.1 Shell
File Edit Shell Debug Options Window Help
===== RESTART: C:/Users/SE_LAB/Desktop/python/if.py =====
나이를 입력하세요 : 15
성인이 아닙니다.
>>>
===== RESTART: C:/Users/SE_LAB/Desktop/python/if.py =====
나이를 입력하세요 : 20
성인입니다.
Ln: 11 Col: 4
```

조건문

조건문 __ 02

▶ 블록 (block)

```
if age >= 19:
```

```
    print("성인입니다.")
```

```
    print("영화관람이 가능합니다.")
```

블록

여러 문장을 묶은 것

문장 앞 동일한 수만큼의 띄어쓰기가 된 문장들의 묶음을 '**블록**'이라 한다.

- 조건이 거짓일 때 연속하여 다른 조건을 검사 (정수 구분하기)

```
if num > 0 :
```

```
    print("양의 정수입니다")
```

```
elif num == 0 :
```

```
    print(" 0 입니다 . ")
```

```
else :
```

```
    print("음의 정수입니다.")
```

해당 조건이 거짓이면

다음 조건을 검사한다

연속하여 다른 조건을 검사하기 위해선 'elif' 를 사용한다 .
elif 는 else if 를 합친 것이다 .

조건문

조건문 __ 02

▶ 블록 실습하기

4 칸 띄고

```
age = int (input("나이를 입력하세요 : "))  
if age >= 19 :  
    print("성인입니다.")  
    print("영화관람이 가능합니다.")  
else :  
    print("성인이 아닙니다.")  
    print("영화관람이 불가능합니다.")
```

조건 (age >= 19)
입력한 나이가 19 세 이상일 경우
성인이므로 영화관람 가능

입력한 나이가 19 세 미만일 경우
성인이 아니므로 영화관람 불가능

```
Python 3.5.1 Shell  
File Edit Shell Debug Options Window Help  
===== RESTART: C:/Users/SE_LAB/Desktop/python/2222.py =====  
나이를 입력하세요 : 15  
성인이 아닙니다.  
영화관람이 불가능합니다.  
>>>  
===== RESTART: C:/Users/SE_LAB/Desktop/python/2222.py =====  
나이를 입력하세요 : 20  
성인입니다.  
영화관람이 가능합니다.  
>>>  
Ln: 13 Col: 4
```

조건문

조건문 __ 02

▶ 실습하기

4 칸 띄고

```
num = int(input("정수를 입력하세요 : "))  
  
if num > 0 :                                # 조건 ( num >= 0 )  
    print("양의 정수입니다.")              # 입력한 정수가 0 이상일 경우  
  
elif num == 0 :                             # 입력한 정수가 0 일 경우  
    print("0입니다.")  
  
else :  
    print("음의 정수입니다.")              # 입력한 정수가 0 이상 또는 0 이 아닐 경우
```

```
Python 3.5.1 Shell  
File Edit Shell Debug Options Window Help  
RESTART: C:/Users/user/AppData/Local/Programs/Python/Python35-32/2.py  
정수를 입력하세요: -2  
음의 정수입니다.  
>>>  
RESTART: C:/Users/user/AppData/Local/Programs/Python/Python35-32/2.py  
정수를 입력하세요: 0  
0입니다.  
>>>  
RESTART: C:/Users/user/AppData/Local/Programs/Python/Python35-32/2.py  
정수를 입력하세요: 2  
양의 정수입니다.  
>>>  
Ln: 15 Col: 4
```


➤ 조건문 실습하기

사용자로부터 시험 점수를 입력 받아,

점수가 **100 점 이면 "A"**

점수가 **70 점 이상이면 "B"**

점수가 **50 점 이상이면 "C"**

그 이외의 점수는 "F"

메시지를 출력하는 조건문을 작성해보자.

조건문

조건문 __ 02

➤ 조건문 실습하기 (답)

```
score = int(input(" 점수를 입력하세요 :"))
```

```
if score == 100:
```

```
    print("A")
```

```
elif score >= 70:
```

```
    print("B")
```

```
elif score >= 50:
```

```
    print("C")
```

```
else :
```

```
    print("F")
```

➤ 논리 연산자 결과 값

논리 연산자를 사용한 수식은 **True(참)** 과 **False(거짓)** 의 두 가지 결과 값을 생성한다.

논리 연산자	뜻
x and y	x와 y가 모두 참이면 참, 그렇지 않으면 거짓
x or y	x와 y중 하나라도 참이면 참, 모두가 거짓이면 거짓
not x	x가 참이면 거짓, x가 거짓이면 참

조건문

조건문 __ 02

➤ 논리 연산자1 (and)

x 와 y 가 모두 참 (true) 이면 참 , 그렇지 않으면 거짓 (false)

토익 점수가 800 점 이상이고 , 학점이 4.0 이상이면 장학금을 받을 수 있다 .

조건 1

조건 2

```
toeic = int (input("토익 점수를 입력하세요 : "))      # 토익 점수 입력 받기
score = float(input("학점을 입력하세요 : "))          # 학점 입력 받기
if toeic >= 800 and score >= 4.0 :                      # 조건 1 and 조건 2
    print("장학금을 받을 수 있습니다.")              # 두 가지 조건 모두 true( 참 ) 일 경우
else :
    print("장학금을 받을 수 없습니다 ..")             # 두 가지 조건 모두 false( 거짓 ) 일 경우
```

조건문

조건문 __ 02

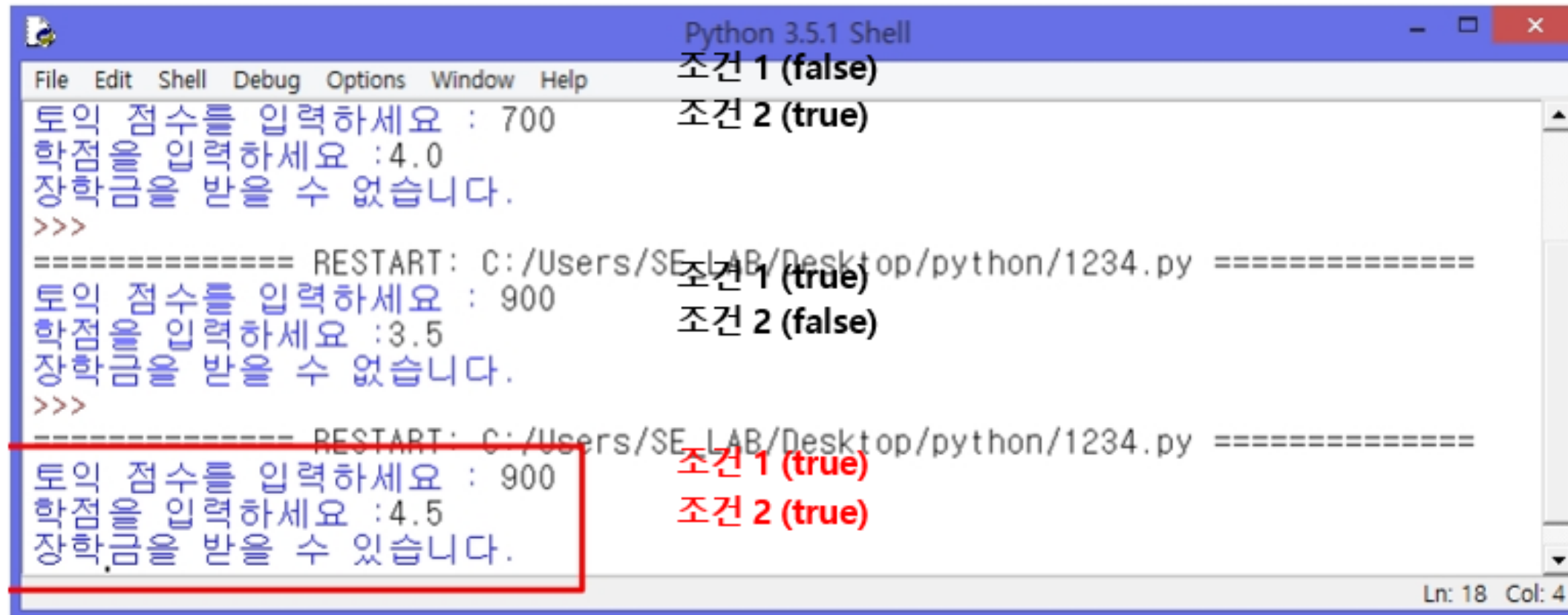
➤ 논리 연산자1 (and)

x 와 y 가 모두 참 (true) 이면 참 , 그렇지 않으면 거짓 (false)

토익 점수가 800 점 이상이고 , 학점이 4.0 이상이면 장학금을 받을 수 있다 .

조건 1

조건 2



```
Python 3.5.1 Shell
File Edit Shell Debug Options Window Help
조건 1 (false)
조건 2 (true)
토익 점수를 입력하세요 : 700
학점을 입력하세요 : 4.0
장학금을 받을 수 없습니다.
>>>
===== RESTART: C:/Users/SE-LAB/Desktop/python/1234.py =====
조건 1 (true)
조건 2 (false)
토익 점수를 입력하세요 : 900
학점을 입력하세요 : 3.5
장학금을 받을 수 없습니다.
>>>
===== RESTART: C:/Users/SE-LAB/Desktop/python/1234.py =====
조건 1 (true)
조건 2 (true)
토익 점수를 입력하세요 : 900
학점을 입력하세요 : 4.5
장학금을 받을 수 있습니다.
Ln: 18 Col: 4
```


조건문

조건문 __ 02

➤ 논리 연산자2 (or)

x 와 y 중 하나라도 참이면 참, 모두 거짓이면 거짓

토익 점수가 800 점 이상이거나, 학점이 4.0 이상이면 장학금을 받을 수 있다.

조건 1

조건 2

```
toeic = int(input("토익 점수를 입력하세요 : "))    # 토익 점수 입력 받기
score = float(input("학점을 입력하세요 : "))       # 학점 입력 받기
if toeic >= 800 or score >= 4.0 :                   # 조건 1 or 조건 2
    print("장학금을 받을 수 있습니다.")            # 두 가지 조건 중 하나라도 참 (true) 일 경우
else :
    print("장학금을 받을 수 없습니다..")           # 두 가지 조건 모두 거짓 (false) 일 경우
```

조건문

조건문 __ 02

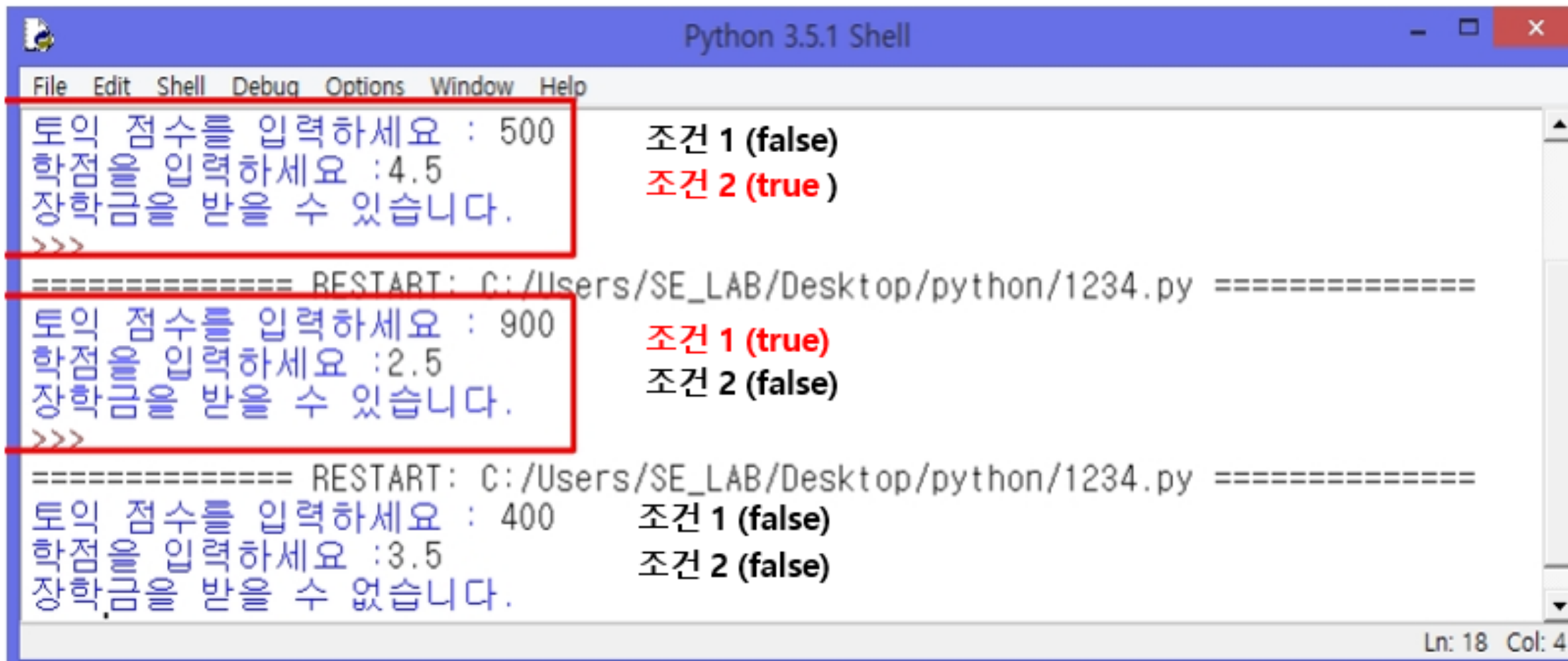
➤ 논리 연산자2 (or)

x 와 y 중 하나라도 참이면 참, 모두 거짓이면 거짓

토익 점수가 800 점 이상이거나, 학점이 4.0 이상이면 장학금을 받을 수 있다.

조건 1

조건 2



```
Python 3.5.1 Shell
File Edit Shell Debug Options Window Help
토익 점수를 입력하세요 : 500
학점을 입력하세요 : 4.5
장학금을 받을 수 있습니다.
>>>
조건 1 (false)
조건 2 (true)

===== RESTART: C:/Users/SE_LAB/Desktop/python/1234.py =====
토익 점수를 입력하세요 : 900
학점을 입력하세요 : 2.5
장학금을 받을 수 있습니다.
>>>
조건 1 (true)
조건 2 (false)

===== RESTART: C:/Users/SE_LAB/Desktop/python/1234.py =====
토익 점수를 입력하세요 : 400
학점을 입력하세요 : 3.5
장학금을 받을 수 없습니다.
>>>
Ln: 18 Col: 4
```

조건문

조건문 __ 02

➤ 논리 연산자3 (not)

x가 참이면 거짓, x가 거짓이면 참

토익 점수가 800 점 이하면, 장학금을 받을 수 없다.

조건

```
toeic = int(input("토익 점수를 입력하세요 : "))  
if not toeic >= 800 :  
    print("장학금을 받을 수 없습니다. ")  
Else :  
    print("장학금을 받을 수 있습니다.")
```

```
# 토익 점수 입력 받기  
# 조건  
  
# 조건이 참 (true) 일 경우  
  
  
# 조건이 거짓 (false) 일 경우
```


조건문

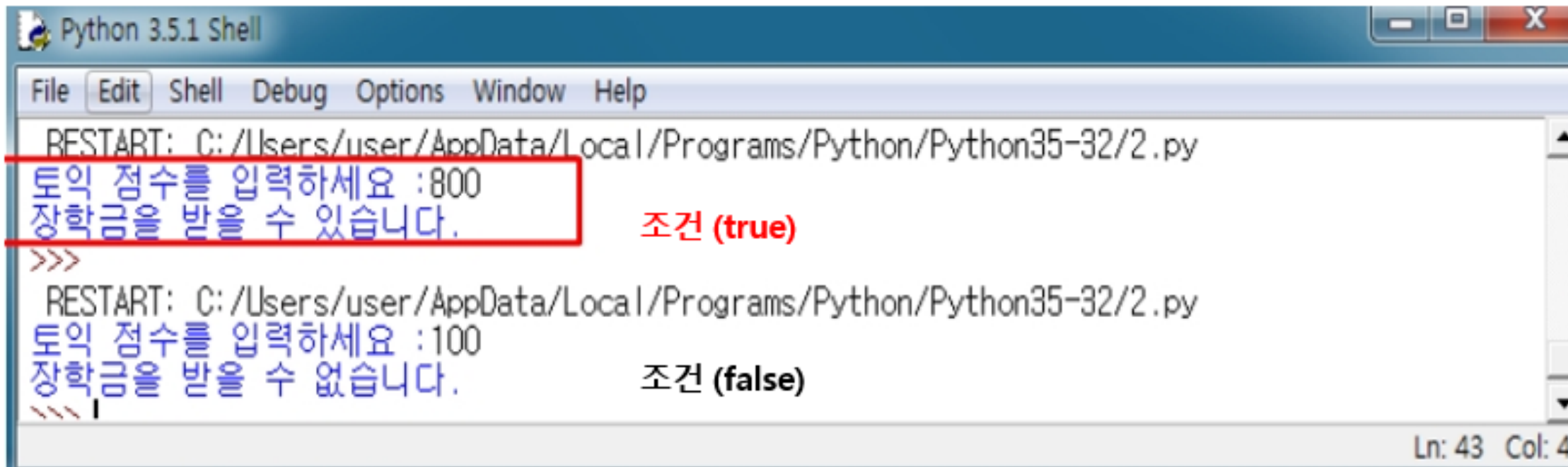
조건문 __ 02

➤ 논리 연산자3 (not)

x가 참이면 거짓, x가 거짓이면 참

토익 점수가 800 점 이하면, 장학금을 받을 수 없다.

조건



```
Python 3.5.1 Shell
File Edit Shell Debug Options Window Help
RESTART: C:/Users/user/AppData/Local/Programs/Python/Python35-32/2.py
토익 점수를 입력하세요 :800
장학금을 받을 수 있습니다.      조건 (true)
>>>
RESTART: C:/Users/user/AppData/Local/Programs/Python/Python35-32/2.py
토익 점수를 입력하세요 :100
장학금을 받을 수 없습니다.      조건 (false)
>>>
```

03

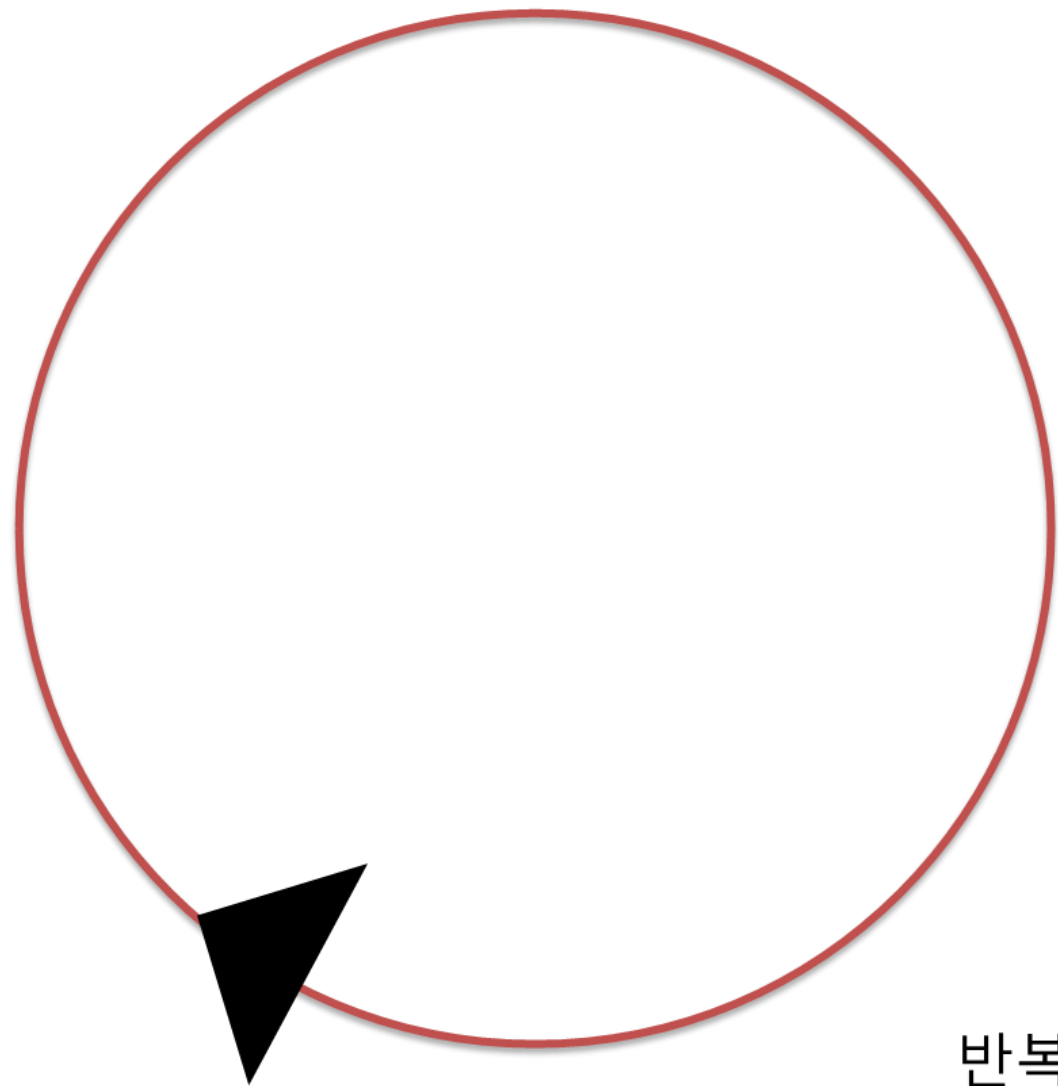
반복문



반복문

▶ 반복문

일련의 작업을 조건에 만족할 때까지 반복 하는 수행문



반복을 루프 (loop) 라고 한다 .

▶ 반복문 종류

- ① for 문
for 문은 반복 횟수가 정해져 있을 때 유용하다.
- ② while 문
while 문은 반복 조건이 정해져 있을 때 유용하다.

▶ 반복문 종류

① for문 -1

정해진 횟수만큼 반복한다.

`range( x )`

0 에서 (x -1) 까지의 숫자를 반환하는 함수

`range()` 함수는 반복 횟수 (정수) 를 자동으로 생성해준다 .

➤ 반복문 종류

① for문

정해진 횟수만큼 반복한다.

0 에서 (반복횟수 - 1) 까지 정수를 자동 생성
↓
for 변수 **in range** (반복횟수):
반복할 문장

➤ 반복문 종류

① for문

정해진 횟수만큼 반복한다.

반복할 때마다 변수 x 에 생성된 값을 대입하면서 문장을 반복한다.

0 에서 9 까지 정수를 자동 생성

for x in range (10):

print("Hello python")

반복할 문장↑

➤ 반복문 종류

① for문

정해진 횟수만큼 반복한다.

```
for x in range(10):  
    print("Hello python")
```

x의 값은 0, print("Hello python") 실행

x의 값은 1, print("Hello python") 실행

x의 값은 2, print("Hello python") 실행

...

x의 값은 9, print("Hello python") 실행

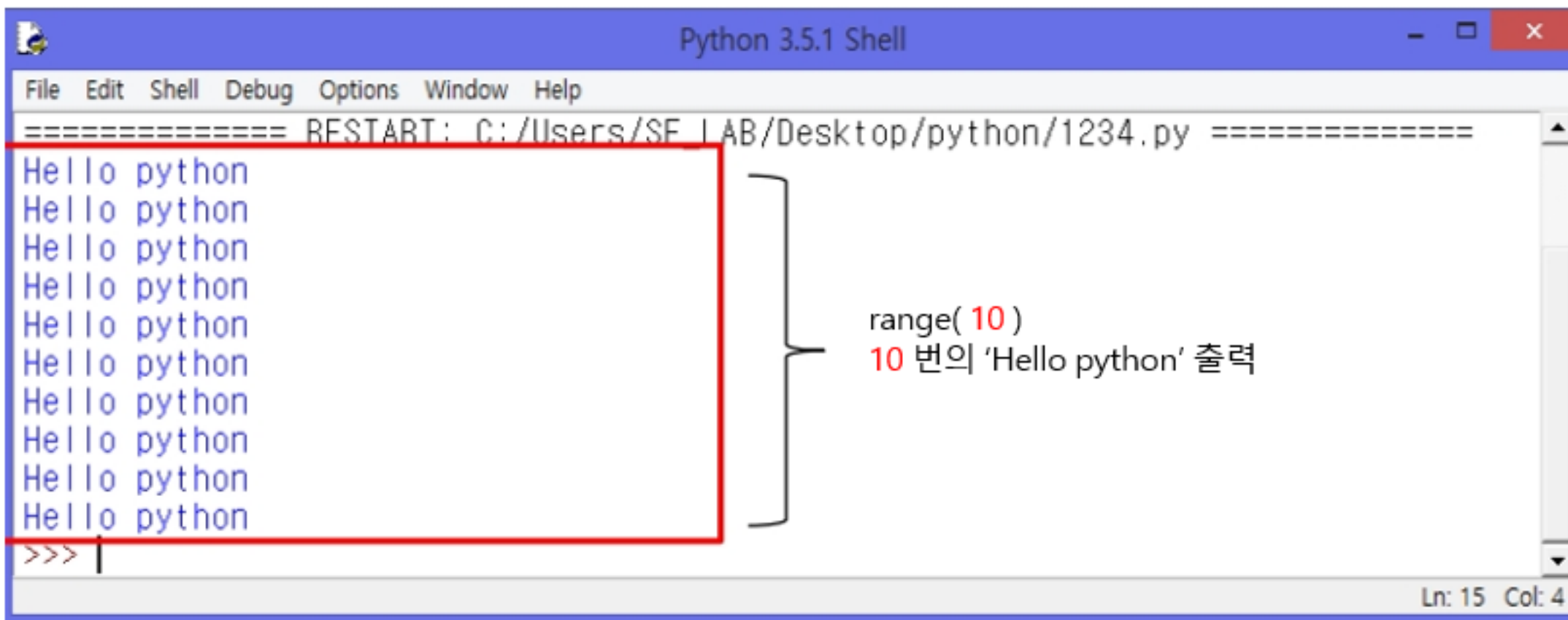
반복문

반복문 __ 03

▶ for문 실습하기

‘Hello python’을 10 번 반복하여 출력하기

```
for x in range (10):      # range(10) 은 0~9 까지의 숫자를 반환한다 .  
    print("Hello!")      # 반복할 내용은 네 칸 띄고 입력
```



```
Python 3.5.1 Shell  
File Edit Shell Debug Options Window Help  
===== RESTART: C:/Users/SEIAB/Desktop/python/1234.py =====  
Hello python  
Hello python  
Hello python  
Hello python  
Hello python  
Hello python  
Hello python  
Hello python  
Hello python  
Hello python  
>>> |
```

range(10)
10 번의 'Hello python' 출력

Ln: 15 Col: 4

▶ 반복문 종류

① for문 -2

정해진 횟수만큼 반복한다.

`range( start, stop, step )`

start 에서 (stop -1) 까지의 step 만큼 숫자를
증가 또는 감소 시켜 반환한다.

➤ 반복문 종류

① for문

정해진 횟수만큼 반복한다.

반복할 때마다 변수 x 에 생성된 값을 대입하면서 문장을 반복한다.

1 에서 9 까지 1 씩 증가시키면서 정수를 자동 생성

for x in range (1, 10, 1):

print("9 곱하기", x , 9*x)

반복할 문장

➤ 반복문 종류

① for문

정해진 횟수만큼 반복한다.

```
for x in range(1, 10, 1):  
    print("9 곱하기", x, 9*x)
```

x의 값은 **1**, print("9 곱하기", **x**, 9***x**) 실행

x의 값은 **2**, print("9 곱하기", **x**, 9***x**) 실행

...

x의 값은 **9**, print("9 곱하기", **x**, 9***x**) 실행

➤ 반복문 종류

② while문

특정한 조건이 만족되면 계속 반복한다.

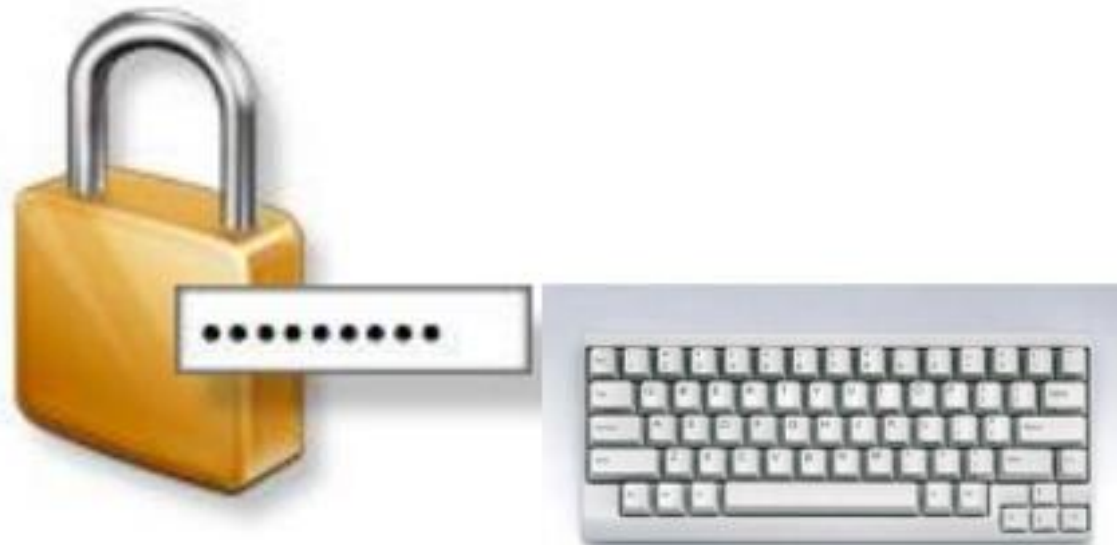
조건이 **true(참)** 이면 반복을 계속한다 .
while 조건 :
반복문장

▶ 반복문 종류

② while문

특정한 조건이 만족되면 계속 반복한다.

비밀번호가 올바르게 입력될 때까지 비밀번호를 입력 받는다 .



➤ while문 실습하기

비밀번호 입력하기

```
set = input(" 설정할 비밀번호를 입력해주세요 .")  
pw = ""  
while (set != pw) :  
    pw = input(" 로그인을 위한 비밀번호를 입력해주세요 .")  
print(" 로그인 성공 ")
```



설정할 비밀번호

설정할 비밀번호와 입력한 비밀번호가 일치할 때까지 반복한다.

설정할 비밀번호와 입력한 비밀번호가 일치하여 로그인에 성공

04 반복문 심화



❖ range 함수

- range(시작숫자, 끝 숫자)
- 시작숫자부터 (끝 숫자-1) 까지의 정수 범위를 표현함



IDLE 종료/재실행 후 다음과 같이 입력 해 봅니다

```
>>> new_list = list(range(0,5))  
>>> print(new_list)
```

- range(정수)
- range(시작숫자, 끝 숫자, 단위)

```
>>> list(range(10))
```

```
>>> list(range(0,1000,50))
```

❖ for 제어문 (반복문)

- for 문은 특정한 문장을 지정된 조건에 따라 반복해 줌
- 오른쪽처럼 프로그래머가 수동으로 하나씩 입력 하는 것은 너무 많은 시간을 소비함
- for 변수명 in range 를 이용하여 손쉽게 출력

```
>>> new_list = [215, 287, 311, 241, 290]
>>> print(new_list[0])
215
>>> print(new_list[1])
287
>>> print(new_list[2])
311
>>> print(new_list[3])
241
>>> print(new_list[4])
290
```



```
>>> for i in [0, 1, 2, 3, 4]:
    print(i)
```

```
>>> for i in range(0, 5):
    print(i)
```



동일한 의미

```
215
287
311
241
290
>>>
```

```
>>> new_list = [215, 287, 311, 241, 290]
>>> for i in new_list:
    print(i)
```

```
i = new_list[i]
```



배열의 길이를 읽어
range(0, 배열길이-1) 까지 반복해줌

Python : 반복문

❖ for 문과 if 문의 복합 사용

- 예시 : 비밀번호 문제

```
password = "1q2w3e4r!"
pass_flag = False

for i in range(5):
    #---- 비밀번호 입력단 ----
    user_id = input("이름을 입력 해 주세요\n")
    user_input = input("비밀번호를 입력 해 주세요\n")
    count = 5-(i+1)

    #---- 검증 ----
    if user_input == password:
        print("%s님 환영합니다." %user_id)
        pass_flag = True
        break #for문 종료

    #---- 오류 ----
    else:
        print("잘못 된 비밀번호입니다. 다시 시도하세요.")

    #---- 입력가능 횟수 ----
    if count>0:
        print("잔여 시도 횟수는 %d 입니다." %count)

if pass_flag == False:
    print("비정상적인 접근입니다.")
```

5회까지 사용자의 비밀번호 입력을 허용

"%s 내용" %변수명으로 문자열에 문자열 변수 삽입 가능

`print(user_id, "님 환영합니다.")`

현재 반복문을 강제 중지시킬 수 있음

For 문이 1회 반복될 때 마다 `i`가 1씩 상승하고, `count` 변수 에 `5 - (i + 1)` 만큼의 값이 적재됨 `count = 4..3..2..1..`

이중 if문 (if 문 안에서 if문 사용)

"%d 내용" %변수명으로 문자열에 정수형 변수 삽입 가능

test3.py - C:/Users/User/AppData

File Edit Format Run Options

New File

Ctrl+N

새로운 python 스크립트를 만들어서
입력 해 봅니다. 다 입력했을 때 F5
버튼으로 Run 가능합니다.

Python : 반복문의 응용

❖ 입력 받은 정수에 대해 약수를 출력하기

```
num = int(input("정수를 입력하세요 : "))  
    #input 으로 입력 받은 숫자 문자열을 int 로 변환 해 줍니다 (casting)  
  
a = 1  
divisor = []  
  
while a <= num :  
    if num % a == 0 :  
        print(a)  
        divisor.append(a)  
        #num을 a로 나눴을때 나머지가 0이면 약수로 인정됩니다  
        #약수가 확인되면 divisor list에 삽입합니다.  
  
        a += 1  
        #a는 1씩 커지고, num과 동일한 값이 되면 중단됩니다  
  
print("%d 의 약수의 개수 :" %num, len(divisor))  
print("약수 :",divisor)
```

66 의 약수의 개수 : 8
약수 : [1, 2, 3, 6, 11, 22, 33, 66]

Python : 반복문의 응용

약수 수정.py - C:/Users/youngshin_han/Desktop/새 폴더/약수 수정.py (3.8.6)

File Edit Format Run Options Window Help

```
num = int(input("정수를 입력하세요 : "))

a = 1
divisor = []

while a <= num :
    if num % a == 0:
        print(a)
        divisor.append(a)

    a += 1

print("%d 의 약수의 개수 :" %num, len(divisor))
print("약수 :", divisor)
```


Thank you

컴퓨팅 사고와 데이터분석 기초
한영신

